

T3: Bidease Reporting API — портирование на PHP

Эталонная реализация: bidease/bidease.py (Python). Цель — портировать функциональность на PHP с сохранением структуры выходных данных. Документ самодостаточен: доступ к Python-коду не требуется. Версия документа: 2026-07-27.

1. Цель проекта

Разработать PHP-библиотеку для работы с Bidease Reporting API (mobile DSP). Библиотека загружает справочник кампаний и собирает дневную статистику (показы, клики, расход) на двух уровнях — кампании и креативы, — плюс строит сводную таблицу-агрегат `admin_audit`.

Выход каждой публичной функции — структурированный массив строк (эквивалент `pandas DataFrame` в Python-эталоне) с фиксированным набором и порядком колонок. Всего 4 публичные функции.

2. Общая логика работы библиотеки

API устроен принципиально проще большинства рекламных API: один синхронный GET-эндпоинт, отдающий CSV. Нет OAuth, нет `submit/poll/download`, нет пагинации, нет перебора сущностей.

- Группировка выполняется на сервере. Разрез задаётся повторяемым параметром `group` (например `group=Day&group=CampaignID`). Поэтому на каждую функцию — ровно один HTTP-запрос на весь период, независимо от числа кампаний и дней.
- Справочников-эндпоинтов нет. Справочник кампаний собирается тем же `stats`-эндпоинтом: запрашиваются группировки-атрибуты за длинный период, метрики отбрасываются, строки дедуплицируются по `campaign_id`.
- Обогащение. К строкам добавляются константные и вычисляемые поля (раздел 3.7), затем колонки приводятся к фиксированному набору и порядку.

Не делать запрос на каждый день и не перебирать кампании — это лишняя нагрузка и другой результат (см. 3.6 про зависимость показов от разреза группировки).

3. Ключевые принципы работы с Bidease Reporting API

3.1. Авторизация

- Статический API-токен, который выдаёт техподдержка / CSM Bidease. Эндпоинтов выпуска или обновления токена не существует — перевыпуск только вручную через поддержку. TTL не документирован (токен долгоживущий).
- Токен передаётся `query`-параметром `api-token` в каждом запросе — не заголовком и не `Bearer`-токеном. Написание канонично через дефис.
- Хранить в переменной окружения `API_TOKEN`; в логах и сообщениях об ошибках токен не выводить (при логировании URL — маскировать значение `api-token`).
- К токену привязана таймзона (по умолчанию UTC+0): все даты и часы запроса и ответа интерпретируются в ней.

3.2. Базовый URL и используемый эндпоинт

BASE_URL = `https://ui-api.bidease.com`

Метод	Путь	Назначение
GET	<code>/api/reporting/v1/stats</code>	Единственный метод API: отчёт с произвольной группировкой

Все четыре функции библиотеки строятся на этом одном эндпоинте — разными комбинациями `group` и периода.

3.3. Схема получения данных

Метод синхронный: данные приходят сразу в теле ответа как `text/csv`. Пагинации нет — весь результат одним телом. `UUID`, `polling` и скачивание файлов отсутствуют как класс.

3.4. Лимиты API

Лимит	Значение	Как соблюдать
Диапазон дат	в пределах 1 года от текущей даты	Валидировать на входе (PERIOD_MAX_DAYS = 365)
Группировок в запросе	не более 7 значений group	Библиотека использует максимум 3
Rate limits	НЕ документированы	Защитный backoff при 429 (см. 3.5)
Пагинация	отсутствует	—

Про rate limits честно: в документации Videase их нет, и на практике 429 ни разу не наблюдался. Область действия (на токен / на аккаунт), размер пула и период обнуления неизвестны. Поэтому backoff в клиенте — не реализация известного лимита, а защита на случай его появления. Если при промышленной нагрузке начнут приходить 429 — уточнять параметры у CSM Videase.

3.5. Обработка ошибок и rate-limit

- HTTP 429 — повторить с экспоненциальным backoff: стартовая пауза 1 сек, удвоение, до 5 повторов. Уважать заголовок Retry-After, если он присутствует.
- HTTP 401 — невалидный токен. Тело ответа ПУСТОЕ — сообщения об ошибке нет, диагностировать только по коду.
- HTTP 400 — тело plain-text: " -> fromdate should be less than todate".
- Тела ошибок — не CSV и не JSON, а короткий plain-text либо пустая строка. Парсер CSV к ним применять нельзя.

Тот же самый текст ошибки 400 приходит и когда fromdate выходит за лимит одного года — сообщение вводит в заблуждение. Поэтому период валидировать на стороне клиента ДО запроса, иначе причина ошибки будет неочевидна.

3.6. Подводные камни формата данных (CSV)

todate — ЭКСКЛЮЗИВНАЯ граница. День todate в отчёт НЕ входит. Это главная ловушка API. Публичные функции принимают date_from / date_to включительно, а в запрос уходит todate = date_to + 1 день. Пример: данные за 21–22 июля → fromdate=2026-07-21&todate=2026-07-23.

Формат значений day — MM/DD/YYYY 00:00:00 (американский порядок + время, НЕ ISO): 07/21/2026 00:00:00. Парсить явным форматом и падать с ошибкой при несовпадении — молчаливый null здесь опаснее исключения, потому что даёт тихо битые даты во всей выгрузке.

- Структура заголовка CSV: сначала 35 колонок метрик, затем колонки группировок — в порядке их указания в запросе, именами в нижнем регистре (Day → day, CampaignID → campaignid, CreativeID → creativeid). Реальный заголовок богаче документации: в нём есть profit, roi, uniq_*, post_*, cr, ipm, roas, esra1-6, а заявленной в доках i2c — нет. Обращаться к колонкам по имени, не по индексу.
- Кодировка: тело — UTF-8, но заголовок Content-Type: text/csv идёт БЕЗ charset. В PHP тело приходит байтами как есть — не применять к нему utf8_encode / mb_convert_encoding «на всякий случай», иначе кириллица в именах кампаний превратится в мусор. Разделитель — запятая, дробная часть чисел — точка.
- Числа приходят с высокой точностью: spend = 17.245259999999991. Приводить к float и округлять до 2 знаков (см. 3.7 и раздел 8).
- Свежие даты (сегодня/вчера) могут «доезжать»: счётчики обновляются на лету. Для устоявшихся дат повторный запрос стабилен.

Показы зависят от РАЗРЕЗА группировки — это свойство API, а не баг. Один и тот же период, три разреза (факт, устоявшиеся данные):

Разрез	21.07	22.07
group=Day	187539	190533
group=Day+CampaignID	187327	190628
group=Day+CampaignID+CreativeID	186732	190676

Разброс 0.08–0.43 %. Клики и расход при этом совпадают во всех разрезах точно. Повторный запрос тем же разрезом даёт идентичные числа — дрейфа нет. Практический вывод: суммы показов между уровнями

(аудит ↔ кампании ↔ креативы) сходиться «до единицы» не обязаны; сверять корректность выгрузки по clicks и spend.

Поведение при пустом результате: запрос С группировками → только строка заголовка, 0 строк данных; запрос БЕЗ группировок → одна строка нулей по всем метрикам. Второе не путать с реальными нулями. Пустой ответ — не ошибка: функция должна вернуть пустой результат с правильным набором колонок.

3.7. Обогащение строк (соглашение проекта)

Константные и вычисляемые поля добавляются на стороне клиента ДО финальной сборки колонок.

Значения констант — примеры, заменяются при интеграции.

- Для всех функций: account_id = 1, source_type_id = 10.
- Справочник кампаний: product_name = "prod_test", camp_type = "camp_test", camp_category = "cat_test", owner_id = 1. Внимание: product_id — РЕАЛЬНЫЙ ProductID из API (не константа, в отличие от других проектов-источников).
- Расход — ровно одно поле costs_usd ← API-поле spend как есть, округление до 2 знаков.
- Составные ключи: id_key_camp = "1_" + campaign_id (все функции); id_key_ad = id_key_camp + "_" + creative_id (только уровень креативов).

Валюта — доллары США. Videase отдаёт расход в USD, поэтому рублёвые поля конвенции (costs_nds, costs_without_nds, costs_nds_ak, costs_without_nds_ak) и константа агентской комиссии ak в этом источнике НЕ применяются и в выгрузке отсутствуют. НДС не считается, курс не применяется — пересчёт валюты и налогов выполняет приёмник данных.

Ключ id_key_ad ДВУХЗВЕННЫЙ: в Videase нет уровня «группа объявлений», иерархия — кампания → креатив.

4. Публичные функции

Колонки и их порядок — фиксированные контракты.

Все stat-функции принимают date_from / date_to (YYYY-MM-DD) ВКЛЮЧИТЕЛЬНО. Валидация до запроса: date_from <= date_to и date_from не старше 365 дней от сегодня. При нарушении — исключение ДО обращения к API.

4.1. get_campaign_dict() — справочник кампаний

Запрос: GET /api/reporting/v1/stats с

group=CampaignID&group=CampaignName&group=AdvertiserID&group=ProductID, период — последний год (fromdate = сегодня - 364 дня, todate = сегодня). Метрики из ответа отбрасываются; строки дедуплицируются по campaign_id (выживает первая), строки без campaign_id отбрасываются.

В справочник попадают только кампании, имевшие хотя бы одно событие за период — следствие того, что справочник собирается из отчёта. Кампании без открутки не видны.

Поле	Тип	Источник
campaign_id	integer	CSV campaignid
campaign_name	string	CSV campaignname
advertiser_id	integer	CSV advertiserid
account_id	integer	константа 1
source_type_id	integer	константа 10
product_id	integer	CSV productid — РЕАЛЬНЫЙ ID продукта
product_name	string	константа "prod_test"
camp_type	string	константа "camp_test"
camp_category	string	константа "cat_test"
id_key_camp	string	вычисляется: "1_" + campaign_id
owner_id	integer	константа 1

4.2. get_campaigns_daily_stat(date_from, date_to) — статистика по кампаниям

Запрос: group=Day&group=CampaignID, один GET на весь период. Гранулярность: одна строка на день × кампанию (уникальность обеспечивает серверная группировка — дедупликация на клиенте не нужна). Строки без campaign_id отбрасываются.

Поле	Тип	Источник
date	string	CSV day (MM/DD/YYYY 00:00:00)
	YYYY-MM-DD	
campaign_id	integer	CSV campaignid
impressions	integer	CSV impressions
clicks	integer	CSV clicks
costs_usd	float	CSV spend, округление до 2 знаков (доллары США)
account_id	integer	константа 1
source_type_id	integer	константа 10
id_key_camp	string	вычисляется: "1_" + campaign_id

Названия кампаний в статистике НЕТ — джойнить из справочника по campaign_id (либо добавить group=CampaignName, но тогда разрез изменится, см. 3.6).

4.3. get_creatives_daily_stat(date_from, date_to) — статистика по креативам

Запрос: group=Day&group=CampaignID&group=CreativeID, один GET на весь период. Гранулярность: одна строка на день × кампанию × креатив. Строки без campaign_id ИЛИ без creative_id отбрасываются (оба нужны для ключа).

Поле	Тип	Источник
date	string	CSV day
campaign_id	integer	CSV campaignid
creative_id	integer	CSV creativeid
impressions	integer	CSV impressions

clicks	integer	CSV clicks
costs_usd	float	CSV spend, round(2)
account_id	integer	константа 1
source_type_id	integer	константа 10
id_key_camp	string	"1_" + campaign_id
id_key_ad	string	id_key_camp + "_" + creative_id

Один креатив может крутиться в нескольких кампаниях — строки при этом разделены по campaign_id, и id_key_ad у них различается. Это корректно.

4.4. get_admin_audit(date_from, date_to) — сводный аудит

Собственного запроса к API нет — это агрегат поверх 4.2:

- Взять результат get_campaigns_daily_stat(date_from, date_to); пустой → вернуть пустой результат, справочник НЕ запрашивать.
- Взять owner_id из get_campaign_dict(), прижойнить по campaign_id.
- Сгруппировать по date × account_id × source_type_id × owner_id, просуммировать impressions, clicks, costs_usd; сумму costs_usd округлить до 2 знаков.
- Добавить chef_flag = 1. Итого 2 HTTP-запроса на вызов.

Кампания может быть в статистике, но отсутствовать в справочнике (период справочника — 364 дня). Тогда owner_id пустой → подставить 1 ДО группировки, иначе строка молча выпадет из агрегата.

Поле	Тип	Источник
date	string	ключ группировки
account_id	integer	константа 1 (ключ группировки)
source_type_id	integer	константа 10 (ключ группировки)
owner_id	integer	join из справочника (пусто → 1; ключ группировки)
impressions	integer	сумма по дню
clicks	integer	сумма по дню
costs_usd	float	сумма по дню, round(2)
chef_flag	integer	константа 1

5. Алгоритм сбора статистики (общий шаблон)

1. Валидировать период: `date_from <= date_to`; `date_from` не старше 365 дней от сегодня. Нарушение -> исключение ДО HTTP-запроса.
2. Собрать query: `api-token`, `fromdate=date_from`, `todate=date_to + 1 день`, затем повторяемые `group=...` (порядок группировок определяет порядок колонок-группировок в CSV).
3. GET `/api/reporting/v1/stats` -> `text/csv`.
429 -> экспоненциальный backoff (1, 2, 4, 8, 16 сек; максимум 5 повторов).
4. Распарсить CSV по ИМЕНАМ колонок заголовка. Пустое тело или только заголовок -> пустой результат с правильным набором колонок (не ошибка).
5. Переименовать колонки-группировки:
`day->date`, `campaignid->campaign_id`, `campaignname->campaign_name`,
`advertiserid->advertiser_id`, `productid->product_id`,
`creativeid->creative_id`.
6. Привести типы: `date (MM/DD/YYYY HH:MM:SS -> YYYY-MM-DD`, явным форматом),
`id-поля -> integer`, `impressions/clicks -> integer`,
`costs_usd <- spend -> float, round(2)`.
7. Отбросить строки без обязательных `id` (см. описание функции).
8. Обогащать: константы, составные ключи (раздел 3.7).
9. Привести к фиксированному набору и порядку колонок.

6. Примеры запросов и ответов API (реальные)

6.1. Форма запроса

```
GET https://ui-api.bidease.com/api/reporting/v1/stats
?api-token={API_TOKEN}
&fromdate=2026-07-21
&todate=2026-07-23      <- эксклюзивно: данные по 22.07 включительно
&group=Day
&group=CampaignID
```

Ответ: HTTP 200, Content-Type: text/csv (без charset!)

6.2. Заголовок CSV (реальный, живой API)

Одна строка; здесь разбита для читаемости. 35 метрик, затем колонки группировок:

```
conversions,spend,profit,roi,impressions,uniq_impressions,clicks,
uniq_clicks,post_clicks,post_views,ctr,ecpm,ecpc,cr,ipm,cpi,roas,ad_roas,
ad_purchases,ad_roas_purchases,revenue,iap_sum,iap,goal1,goal2,goal3,goal4,
goal5,goal6,есра1,есра2,есра3,есра4,есра5,есра6,day,campaignid
```

Библиотеке нужны только `spend`, `impressions`, `clicks` и колонки группировок. Остальные метрики (включая расчётные `ctr`, `ecpm`, `ecpc`, `cpi` и `conversions / revenue / iap* / goal1-6`) в выгрузку не берутся — решение проекта.

6.3. Статистика по кампаниям (group=Day&group=CampaignID)

Строка данных целиком, как её отдаёт API:

```
2,17.24525999999991,0,0,5788,2766,406,0,0,0,0.07014512785072564,
2.9794851416724097,0.04247600985221653,0,0.346,8.622629999999955,0,0,0,0,
17.24525999999991,0,0,2,1,0,0,0,0,8.622629999999955,17.24525999999991,0,0,0,0,
07/22/2026 00:00:00,154402
```

Читается так: `spend = 17.24525999999991` → `costs_usd = 17.25`; `impressions = 5788`; `clicks = 406`; `day = 07/22/2026 00:00:00` → `date = 2026-07-22`; `campaignid = 154402`.

6.4. Справочник (group=CampaignID+CampaignName+AdvertiserID+ProductID)

Хвост строки — колонки группировок в порядке запроса:

```
...,154369,x5_x5_igronik_Пятерочка_МояВыгода_android,2608,29153
...,154402,x5_x5_igronik_Пятерочка_МояВыгода_ios,2608,29154
```

campaignid=154369, campaignname=..._android (кириллица, UTF-8), advertiserid=2608, productid=29153.

6.5. Креативы (group=Day+CampaignID+CreativeID)

```
...,07/22/2026 00:00:00,154369,647446
...,07/21/2026 00:00:00,154402,647448
```

6.6. Ошибки (реальные ответы)

HTTP 400, тело (plain-text): " -> fromdate should be less than todate"
Причина 1: fromdate >= todate
Причина 2: fromdate старше 1 года <- ТОТ ЖЕ текст (вводит в заблуждение)

HTTP 401, тело: "" (пусто)
Причина: невалидный api-token

7. Примеры таблиц на выходе

Реальные строки, период 21–22.07.2026.

7.1. get_campaign_dict — справочник кампаний

campaign_id	campaign_name	advertiser_id	product_id
154369	x5_x5_igronik_Пятерочка_МояВыгода_android	2608	29153
154402	x5_x5_igronik_Пятерочка_МояВыгода_ios	2608	29154

Константные и вычисляемые поля тех же двух строк:

account_id	source_type_id	product_name	camp_type	camp_category	id_key_camp	owner_id
1	10	prod_test	camp_test	cat_test	1_154369	1
1	10	prod_test	camp_test	cat_test	1_154402	1

7.2. get_campaigns_daily_stat — статистика по кампаниям

date	campaign_id	impressions	clicks	costs_usd	account_id	source_type_id	id_key_camp
2026-07-21	154369	179576	7063	531.86	1	10	1_154369
2026-07-21	154402	7751	519	23.09	1	10	1_154402
2026-07-22	154369	184840	7006	553.76	1	10	1_154369
2026-07-22	154402	5788	406	17.25	1	10	1_154402

7.3. get_creatives_daily_stat — статистика по креативам

Колонки account_id = 1 и source_type_id = 10 опущены для компактности — в выгрузке они присутствуют.

date	campaign_id	creative_id	impressions	clicks	costs_usd	id_key_ad
2026-07-21	154369	647445	161895	6032	480.95	1_154369_647445
2026-07-21	154369	647446	17085	1031	50.91	1_154369_647446
2026-07-21	154402	647447	6337	401	18.88	1_154402_647447
2026-07-21	154402	647448	1415	118	4.21	1_154402_647448
2026-07-22	154369	647446	20743	1157	61.81	1_154369_647446

7.4. get_admin_audit — сводный аудит

date	account_id	source_type_id	owner_id	impressions	clicks	costs_usd	chef_flag
2026-07-21	1	10	1	187327	7582	554.95	1
2026-07-22	1	10	1	190628	7412	571.01	1

Проверка сходимости на этих данных: аудит за 21.07 = 179576 + 7751 = 187327 показов и 531.86 + 23.09 = 554.95 USD — совпадает с 7.2 точно. А вот сумма показов по креативам (7.3) за 21.07 даёт 186732 — на 595 меньше: это описанное в 3.6 свойство API, НЕ ошибка выгрузки.

8. Рекомендации по реализации на PHP

- Версия: PHP >= 8.1, Composer-проект (типизированные свойства, enum, readonly).
- HTTP-клиент: Guzzle (или Symfony HttpClient) с retry-middleware на 429; таймаут запроса — 30 сек.

- Повторяемые query-параметры: group передаётся несколько раз. Собирать строку запроса вручную (group=Day&group=CampaignID) либо через `\GuzzleHttp\Psr7\Query::build()` со списком пар. Проверить, что клиент НЕ схлопывает одноимённые параметры в один.
- Парсинг CSV: league/csv либо fgetcsv по потоку. Читать по именам колонок из первой строки, не по индексам (реальный заголовок шире документации и может расширяться).
- Кодировка: тело — UTF-8; никаких перекодировок не применять (см. 3.6).
- Даты: `DateTimeImmutable::createFromFormat('m/d/Y H:i:s', $v)` — и обязательно проверять результат на false, а также `DateTime::getLastErrors()`: молчаливо принятая битая дата хуже исключения. Эксклюзивный todate: `(new DateTimeImmutable($dateTo))->modify('+1 day')->format('Y-m-d')`.
- Пустые ответы: пустое тело и «только заголовок» — штатный результат; возвращать пустой массив с корректным набором колонок, не бросать исключение.
- Логирование: PSR-3. Все пропуски строк (без campaign_id / creative_id) — уровень warning, процесс не прерывать. При логировании URL маскировать api-token.

Деньги — главное расхождение платформ. Python-эталон округляет через numpy: режим half-to-even по двоичному float, поэтому `round(12.345, 2) = 12.34`, тогда как обычный PHP `round()` — half-up и даст 12.35. Использовать `round($v, 2, PHP_ROUND_HALF_EVEN)`. Полной бит-в-бит идентичности на «ровных половинках» это всё равно не гарантирует (PHP применяет pre-rounding), но расхождение остаётся в пределах одного цента и только на редких значениях — при сверке с эталоном использовать допуск ± 0.01 . Для внутреннего учёта деньги лучше хранить в decimal-типе БД (`numeric(18,2)`), а не во float.

Константы (не хардкодить в коде вызовов):

Константа	Значение
HTTP_TIMEOUT_SEC	30
PERIOD_MAX_DAYS	365
MAX_GROUPS	7
RATE_LIMIT_BASE_SEC	1
RATE_LIMIT_RETRY_MAX	5
DICT_PERIOD_DAYS	364 (период справочника)

Тесты: моки HTTP на все 4 функции; пустое тело и «только заголовок»; строки без campaign_id / creative_id; невалидный период (исключение ДО запроса); битый и пустой spend → costs_usd = 0.0 без потери строки; повтор при 429.

9. Критерии приёмки

- [] Все 4 публичные функции возвращают данные с теми же колонками и в том же порядке, что в Python-эталоне bidease.py (разделы 4.1–4.4).
- [] todate передаётся эксклюзивно: вызов за 21–22.07 уходит в API как `fromdate=2026-07-21&today=2026-07-23` и возвращает ровно два дня.
- [] date — строки YYYY-MM-DD; исходный MM/DD/YYYY 00:00:00 распарсен явным форматом, при несовпадении формата — исключение, а не пустое значение.
- [] Расход отдаётся ЕДИНСТВЕННЫМ полем costs_usd (доллары США, 2 знака); полей costs_nds / costs_without_nds / costs*_ak / ak в выгрузке НЕТ.
- [] id_key_camp = "1_" + campaign_id; id_key_ad = id_key_camp + "_" + creative_id (двухзвенный, без уровня группы).
- [] get_admin_audit: суммы clicks и costs_usd совпадают с суммами get_campaigns_daily_stat по тому же дню; кампания, отсутствующая в справочнике, не теряется (owner_id = 1).
- [] Пустой период → пустой результат с правильными колонками, без исключений.
- [] Невалидный период (date_to < date_from; выход за 365 дней) → исключение ДО HTTP-запроса.
- [] Кириллица в campaign_name не искажена (проверять на реальном ответе).
- [] Токен не попадает в логи и сообщения об ошибках.
- [] README с примерами вызова + .env.example (API_TOKEN).